Rockchip RK3588 Linux Robot SDK 快速入门

文档标识：RK-JC-YF-H52

发布版本：V0.0.2

日期：2025-12-05

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有© 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文主要描述了 RK3588 Linux Robot SDK 的基本使用方法，旨在帮助开发者快速了解并使用 SDK 开发包。

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

各芯片系统支持状态

芯片名称	Kernel 版本	Debian 版本	Buildroot 版本
RK3588 系列芯片	6.1	11	2024.02

修订记录

日期	版本	作者	修改说明
2025-10-20	V0.0.1	WJL	Alpha 版本。
2025-12-05	V0.0.2	WJL	增加 Robotic EVB 说明。

目录

Rockchip RK3588 Linux Robot SDK 快速入门

1. 预编译固件
2. 开发环境搭建
   - 2.1 准备开发环境
   - 2.2 安装库和工具集
     - 2.2.1 检查和升级主机的 `python` 版本
     - 2.2.2 检查和升级主机的 `make` 版本
     - 2.2.3 检查和升级主机的 `lz4` 版本
   - 2.3 容器环境搭建
3. 软件开发指南
   - 3.1 开发向导
   - 3.2 芯片资料
   - 3.3 Buildroot 开发指南
   - 3.4 Debian 开发指南
   - 3.5 RKNPU 开发指南
   - 3.6 RK182X 开发指南
   - 3.7 RT-Linux 开发指南
   - 3.8 AMP 开发指南
   - 3.9 EtherCAT 开发指南
   - 3.10 软件更新记录
4. 硬件开发指南
5. SDK 配置框架说明
   - 5.1 SDK 工程目录介绍
   - 5.2 RKROS 工程目录介绍
6. SDK 编译说明
   - 6.1 SDK 编译命令查看
   - 6.2 SDK 板级配置
   - 6.3 SDK 定制化配置
   - 6.4 全自动编译
   - 6.5 各模块编译
     - 6.5.1 U-Boot 编译
     - 6.5.2 Kernel 编译
     - 6.5.3 Recovery 编译
     - 6.5.4 Debian 编译
     - 6.5.5 RKROS 编译
   - 6.6 固件的打包
7. 刷机说明
   - 7.1 Windows 刷机说明
   - 7.2 Linux 刷机说明
   - 7.3 系统分区说明

1. 预编译固件

为帮助开发者快速完成相关评估、对比、开发，我们提供了 RK3588 Robot 开发板预编译固件。

预编译固件可从网盘 预编译固件 获取。
Linux6.1 固件路径：**Robot-> Linux6.1 -> RK3588**

如需修改 SDK 代码，请参考后续章节。

2. 开发环境搭建

2.1 准备开发环境

推荐使用 Ubuntu22.04 及以上版本的系统进行编译，其他的 Linux 发行版需要对软件包做相应调整。

软件要求：Ubuntu22.04 及以上版本。
硬件要求：磁盘空间大于 100GB。进行多个构建，则需要更大的磁盘空间。

2.2 安装库和工具集

通过 apt 命令安装编译 SDK 需要的库和工具集：

```
sudo apt install -y \
bash-completion bc bison bsdmainutils build-essential ccache chrpath cmake \
cpio curl device-tree-compiler diffstat dos2unix expat expect fdisk file \
flex gawk g++-multilib gcc-arm-none-eabi gcc-multilib genext2fs gettext \
git iputils-ping libegl1-mesa libelf-dev libgmp-dev libgucharmap-2-90-dev \
liblz4-tool libmpc-dev libncurses-dev libsdl1.2-dev libssl-dev libtinfo5 \
live-build locales mesa-common-dev mtd-utils mtools net-tools patchelf \
python2 python2-dev python3 python3-git python3-jinja2 python3-pexpect \
python3-pip python3-pyelftools python3-subunit python-is-python3 \
qemu-user-static repo rsync ruby scons socat ssh strace sudo swig \
texinfo time tree u-boot-tools unzip vim
```

> **说明：**
> 上述命令适用于 Ubuntu22.04，其他版本请根据安装包名称采用对应的安装命令，若编译遇到报错，可以视报错信息，安装对应的软件包。
>
> - python 要求安装 3.6 及以上版本，此处以 python 3.6 为例。
> - make 要求安装 4.0 及以上版本，此处以 make 4.2 为例。
> - lz4 要求安装 1.7.3 及以上版本。

2.2.1 检查和升级主机的 `python` 版本

- 检查主机 `python` 版本

```
$ python3 --version
Python 3.10.6
```

- 升级 `python 3.6.15` 新版本

```
PYTHON3_VER=3.6.15
echo "wget
https://www.python.org/ftp/python/${PYTHON3_VER}/Python-${PYTHON3_VER}.tgz"
echo "tar xf Python-${PYTHON3_VER}.tgz"
echo "cd Python-${PYTHON3_VER}"
echo "sudo apt-get install libsqlite3-dev"
echo "./configure --enable-optimizations"
echo "sudo make install -j8"
```

2.2.2 检查和升级主机的 `make` 版本

- 检查主机 `make` 版本

```
$ make -v
GNU Make 4.2
Built for x86_64-pc-linux-gnu
```

- 升级 `make 4.2` 新版本

```
$ sudo apt update && sudo apt install -y autoconf autopoint
git clone https://gitee.com/mirrors/make.git
cd make
git checkout 4.2
git am $BUILDROOT_DIR/package/make/*.patch
autoreconf -f -i
./configure
make make -j8
sudo install -m 0755 make /usr/bin/make
```

2.2.3 检查和升级主机的 `lz4` 版本

- 检查主机 `lz4` 版本

```
$ lz4 -v
*** LZ4 command line interface 64-bits v1.9.3, by Yann Collet ***
```

- 升级 `lz4` 新版本

```
git clone https://gitee.com/mirrors/LZ4_old1.git
cd LZ4_old1
make
sudo make install
sudo install -m 0755 lz4 /usr/bin/lz4
```

2.3 容器环境搭建

为帮助开发者快速完成编译环境准备，我们提供了容器镜像以及镜像制作方法，可从网盘 Docker 镜像获取。

- docker-sdk：基于 amd64/ubuntu22.02 构建，用于编译通用 Linux 固件。
- docker-ros：基于 arm64v8/debian11 构建，用于跨架构编译 ROS 软件包。

Docker 使用说明可以参考文档：

```
<SDK>/docs/cn/Linux/Docker/
└── Rockchip_Developer_Guide_Linux_Docker_Deploy_CN.pdf
```

3. 软件开发指南

3.1 开发向导

为帮助开发者快速上手 SDK 的开发调试，可参考文档：

```
<SDK>/docs/cn/RK3588/
└── Rockchip_Developer_Guide_Linux_Software_CN.pdf
```

3.2 芯片资料

为帮助开发者快速上手 RK3588、RK3588S 的开发调试，可参考文档：

```
<SDK>/docs/cn/RK3588/Datasheet/
├── Rockchip_RK3588_Datasheet_V2.0-20250425.pdf
...
└── Rockchip_RK3588S2_Datasheet_V1.2-20241212.pdf
```

3.3 Buildroot 开发指南

为帮助开发者快速上手 Buildroot 系统的开发调试，可参考文档：

```
<SDK>/docs/cn/Linux/System/
└── Rockchip_Developer_Guide_Buildroot_CN.pdf
```

3.4 Debian 开发指南

为帮助开发者快速上手 Debian 系统的开发调试，可参考文档：

```
<SDK>/docs/cn/Linux/System/
└── Rockchip_Developer_Guide_Debian_CN.pdf
```

3.5 RKNPU 开发指南

SDK 提供了 RKNPU 开发工具：

- RKNN-TOOLKIT2：PC 上进行 RKNN 模型生成以及评估的开发套件，可从 `<SDK>/external/rknn-toolkit2` 目录获取。
- RKNN API：用于推理 RKNN-TOOLKIT2 生成的 RKNN 模型，可从 `<SDK>/external/rknpu2` 目录获取。

具体使用说明，可参考文档：

```
<SDK>/external/rknn-toolkit2/doc/
├── 01_Rockchip_RKNPU_Quick_Start_RKNN_SDK_V2.3.2_CN.pdf
...
├── 04_Rockchip_RKNPU_API_Reference_RKNNRT_V2.3.2_CN.pdf
└── 05_RKNN_Compiler_Support_Operator_List_V2.3.2.pdf
```

3.6 RK182X 开发指南

RK182X 系列协处理器具备高算力、高带宽、低延时等特点，专注于高性能神经网络推理任务。支持 PyTorch、ONNX 等深度学习框架的模型转换、推理和性能评估。

开发者可联系业务获取相关 SDK。

3.7 RT-Linux 开发指南

对实时性能要求较高的产品，可以使用 PREEMPT_RT 或者 Xenomai 方案来提高实时性能。
具体使用说明，可参考文档：

```
<SDK>/docs/Patches/Real-Time-Performance/
├── Rockchip_Develop_Guide_Linux_RealTime_CN.md
├── Rockchip_Develop_Guide_Linux_RealTime_Performance_Test_Report_CN.pdf
└── Rockchip_Trouble_Shooting_Linux_Real-Time_Performance_CN.pdf
```

3.8 AMP 开发指南

对实时性能有更高要求的产品，可以考虑使用 AMP 方案，可参考文档：

```
<SDK>/docs/cn/Common/AMP/
└── Rockchip_Developer_Guide_AMP_CN.pdf
```

3.9 EtherCAT 开发指南

为帮助开发者快速上手 EtherCAT 主站协议的开发调试，可参考文档：

```
<SDK>/docs/cn/Linux/ApplicationNote/
└── Rockchip_Use_Guide_Linux_EtherCAT_IgH_CN.pdf
```

3.10 软件更新记录

软件发布版本可以通过工程 xml 进行查看，具体方法如下：

```
.repo/manifests$ realpath rk3588_linux6.1_robot_release.xml
# 例如:打印的版本号为v0.0.2，更新时间为20251205
# <SDK>/.repo/manifests/release/rk3588_linux6.1_robot_alpha_v0.0.2_20251205.xml
```

软件发布版本升级更新内容通过工程文本可以查看，参考工程目录：

```
<SDK>/.repo/manifests/RK3588_Linux6.1_Robot_SDK_Release_Note.md
```

4. 硬件开发指南

RK3588 硬件设计和用户指南，可参考文档：

```
<SDK>/docs/cn/RK3588/Hardware/
├── RK3588_PinOut_V1.1_20220922.xlsx
...
├── Rockchip_RK3588_EVB1_User_Guide_V1.2_CN.pdf
├── Rockchip_RK3588_EVB7_User_Guide_V1.0_CN.pdf
└── Rockchip_RK3588_Hardware_Design_Guide_V1.4_CN.pdf
```

5. SDK 配置框架说明

5.1 SDK 工程目录介绍

SDK 工程目录包含有 debian、device、docs、kernel、robot、u-boot 等目录。采用 manifest 来管理仓库，用 repo 工具来管理每个目录或其子目录会对应的 git 工程。

- buildroot：基于 Buildroot 2024.02 开发的根文件系统。
- debian：基于 Debian bullseye 开发的根文件系统。
- device/rockchip：存放芯片板级配置以及一些编译和打包固件的脚本和文件。
- docs：存放开发指导文件、平台支持列表、工具使用文档、Linux 开发指南等。
- external：存放第三方相关仓库，包括显示、音视频、摄像头、网络、安全等。
- hal: 存放基于 RK HAL 硬件抽象层的裸机开发库，用于 AMP 方案。
- kernel：存放 Kernel 开发的代码。
- output：存放每次生成的固件信息、编译信息、XML、主机环境等。
- prebuilts：存放交叉编译工具链。
- rkbin：存放 Rockchip 相关二进制和工具。
- robot：存放机器人专用的加速算法库、模型库、rkros 代码等。
- rockdev：存放编译输出固件,实际软链接到 `output/firmware`。
- rtos：存放基于 RT-Thread 4.1 开发的根文件系统。
- tools：存放 Linux 和 Window 操作系统下常用工具。
- u-boot：存放 U-Boot 代码。
- uefi：存放基于 edk2 V2.7 版本进行开发的 UEFI 代码。

5.2 RKROS 工程目录介绍

RKROS 工程位于 `<SDK>/robot/` 目录下，包含 common_algorithm、docs、model_zoo、prebuilts、rkros 等。具体工程说明如下：

- common_algorithm：存放硬件加速算法库，支持 PCL、IDC 等函数接口。
- docs：存放 RKROS 相关的开发文档。
- model _zoo：存放机器人常用的模型库。
- prebuilts：存放预编译的软件包。
- rkros：存放 rkros 代码，是 rkros 的默认工作空间。
- scripts：存放编译、打包 rkros 的脚本。

6. SDK 编译说明

SDK 可通过 `./build.sh` 加目标参数进行相关功能的配置和编译。
具体参考 `<SDK>/device/rockchip/common/README.md` 的编译说明。

6.1 SDK 编译命令查看

通过运行 `./build.sh <target>` 来编译相关功能，可通过 `./build.sh help` 查看具体编译命令。

```
############### Rockchip Linux SDK ###############

Manifest: rk3588_linux6.1_robot_alpha_v0.0.2_20251205.xml

Log colors: message notice warning error fatal

Usage: build.sh [OPTIONS]
Available options:
chip[:<chip>[:<config>]]            choose chip
defconfig[:<config>]                choose defconfig
config                              modify SDK defconfig
...
updateimg                           build update image
otapackage                          build A/B OTA update image
all                                 build images
release                             release images and build info
save                                alias of release
all-release                         build and release images
allsave                             alias of all-release
shell                               setup a shell for developing
cleanall                            cleanup
clean[:module[:module]]...          cleanup modules
post-rootfs <rootfs dir>            trigger post-rootfs hook scripts
help                                usage

Default option is 'all'.

Available options for RKROS:

rkros[:debug|:release]                  build all rkros packages
rkros[:cleanall]                        cleanup all rkros packages
rkros[:<pkg>-[debug|release]]           build the specified rkros package
  supported rkros packages:
...
```

6.2 SDK 板级配置

进入工程 `<SDK>/device/rockchip/rk3588` 目录：

板级配置	说明
rockchip_rk3588_evb1_lp4_v10_defconfig	适用于 RK3588 EVB1 搭配 LPDDR4 开发板
rockchip_rk3588_evb7_v11_defconfig	适用于 RK3588 EVB7 搭配 LPDDR4 开发板
rockchip_rk3588_evb10_v10_defconfig	适用于 RK3588 EVB10 搭配 LPDDR4 开发板
rockchip_rk3588_robotic_evb_v11_defconfig	适用于 RK3588 Robotic EVB 搭配 LPDDR4 开发板
rockchip_rk3588s_evb1_lp4x_v10_defconfig	适用于 RK3588S EVB1 搭配 LPDDR4 开发板
rockchip_defconfig	默认配置, 会软链接到一个具体的板级配置

方法一：

`./build.sh` 后面加上板级配置文件，例如：

选择**适用于 RK3588 Robotic EVB 搭配 LPDDR4 开发板**的板级配置：

```
./build.sh device/rockchip/rk3588/rockchip_rk3588_robotic_evb_v11_defconfig
```

方法二：

`./build.sh lunch` 选择配置

```
./build.sh lunch
Pick a defconfig:

1. rockchip_defconfig
...
7. rockchip_rk3588_robotic_evb_v11_defconfig
...
Which would you like? [1]:
```

6.3 SDK 定制化配置

SDK 可通过 `./build.sh menuconfig` 进行相关配置，目前可配组件主要如下:

```
(rockchip_rk3588_robotic_evb_v11_defconfig) Name of defconfig to save
[*] Rootfs (Buildroot|Debian|Yocto)  --->
[*] Loader (U-Boot)  --->
[ ] AMP (Asymmetric Multi-Processing System)
[*] Kernel (Embedded in an Android-style boot image)  --->
    Boot (Android-style boot image)  --->
[*] Recovery (based on Buildroot)  --->
    *** Security feature depends on buildroot rootfs ***
    Extra partitions (oem, userdata, etc.)  --->
    Firmware (partition table, misc image, etc.)  --->
[*] Update (Rockchip update image)  --->
    Others configurations  --->
```

- Rootfs：根文件系统，在这里可选择 Buildroot、Debian 等不同的根文件系统配置。
- Loader (U-Boot)：引导加载器的配置，通常是 u-boot，用于初始化硬件并加载主操作系统。
- AMP：多核异构启动方案，适用于需要实时性能的应用场景。
- Kernel：这里配置内核选项，定制适合自己的硬件和应用需求的 Linux 内核。
- Boot：这里配置 Boot 分区支持格式。
- Recovery (based on Buildroot)：这是基于 Buildroot 的 recovery 环境的配置，用于系统恢复和升级。
- PCBA test (based on Buildroot)：这是基于 Buildroot 的 PCBA（印刷电路板组装）测试环境的配置。
- Security：安全功能开启， 包含 Secureboot 开启方式、Optee 存储方式、烧写 Key 等。
- Extra partitions：用于配置额外的分区。
- Firmware：在这里配置固件相关选项。
- Update (Rockchip update image)：用于配置 Rockchip 完整固件的选项。
- Others configurations：其他额外的配置选项。

通过 `./build.sh menuconfig` 配置界面提供了一个基于文本的用户界面来选择和配置各种选项。配置完成后，使用 `./build.sh savedefconfig` 命令保存这些配置，这样定制化编译就会根据这些设置来进行。

通过 menuconfig ，可选择不同 Rootfs、Loader、Kernel 等配置，进行各种定制化编译，这样就可以灵活地选择和配置系统组件以满足特定的需求。

6.4 全自动编译

为了确保软件开发套件（SDK）的每次更新都能顺利进行，建议在更新前清理之前的编译产物。这样做可以避免潜在的兼容性问题或编译错误，因为旧的编译产物可能不适用于新版本的SDK。要清理这些编译产物，可以直接运行命令 ./build.sh cleanall。

进入工程根目录执行以下命令自动完成所有的编译：

```
./build.sh all # 只编译模块代码（u-Boot，kernel，Rootfs，Recovery）
               # 需要再执行`./build.sh ./mkfirmware.sh 进行固件打包

./build.sh     # 编译模块代码（u-Boot，kernel，Rootfs，Recovery）
               # 打包成 update.img 完整升级包
               # 所有编译信息复制和生成到 out 目录下
```

6.5 各模块编译

6.5.1 U-Boot 编译

```
./build.sh uboot
```

6.5.2 Kernel 编译

方法一：

```
./build.sh kernel
```

方法二：

```
cd kernel
export CROSS_COMPILE=../prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-
make ARCH=arm64 rockchip_linux_defconfig rk3588.config rk3588_linux.config rk3588_robotic.config
make ARCH=arm64 rk3588-robotic-evb-v11-linux.img -j
```

方法三：

```
cd kernel
export CROSS_COMPILE=aarch64-linux-gnu-
make ARCH=arm64 rockchip_linux_defconfig rk3588.config rk3588_linux.config rk3588_robotic.config
make ARCH=arm64 rk3588-robotic-evb-v11-linux.img -j
```

6.5.3 Recovery 编译

进入工程根目录执行以下命令自动完成 Recovery 的编译及打包。

```
./build.sh recovery
```

编译后在 Buildroot 目录 `<SDK>/output/rockchip_rk3588_recovery/images` 生成 recovery.img。

> 注： recovery.img 包含 kernel，所以每次 kernel 修改，recovery.img 需要重新打包生成。
> recovery.img 重新打包的方法如下：

```
source buildroot/envsetup.sh
cd buildroot
make recovery-reconfigure
cd -
./build.sh recovery
```

> 注：Recovery 是非必需的功能，有些板级配置不会设置

6.5.4 Debian 编译

```
./build.sh debian
```

编译后在 debian 目录下生成 linaro-rootfs.img。

> **说明：需要预先安装相关依赖包**
> sudo apt-get install binfmt-support qemu-user-static live-build
> sudo dpkg -i ubuntu-build-service/packages/*
> sudo apt-get install -f

6.5.5 RKROS 编译

编译 rkros 工作空间下的所有代码：

```
./build.sh rkros:debug
或
./build.sh rkros:release
```

编译 rkros 工作空间下指定的代码：

```
./build.sh rkros:rkros_examples-debug
或
./build.sh rkros:rkros_examples-release
```

> **debug 方式**：未指定 install_base，编译产物在 `<SDK>/robot/rkros/install` 目录下。
>
> **release 方式**：标准化安装到 `/opt/rkros/packagename`，自动打包成 deb，编译产物在 `<SDK>/robot/rkros/output` 目录下。

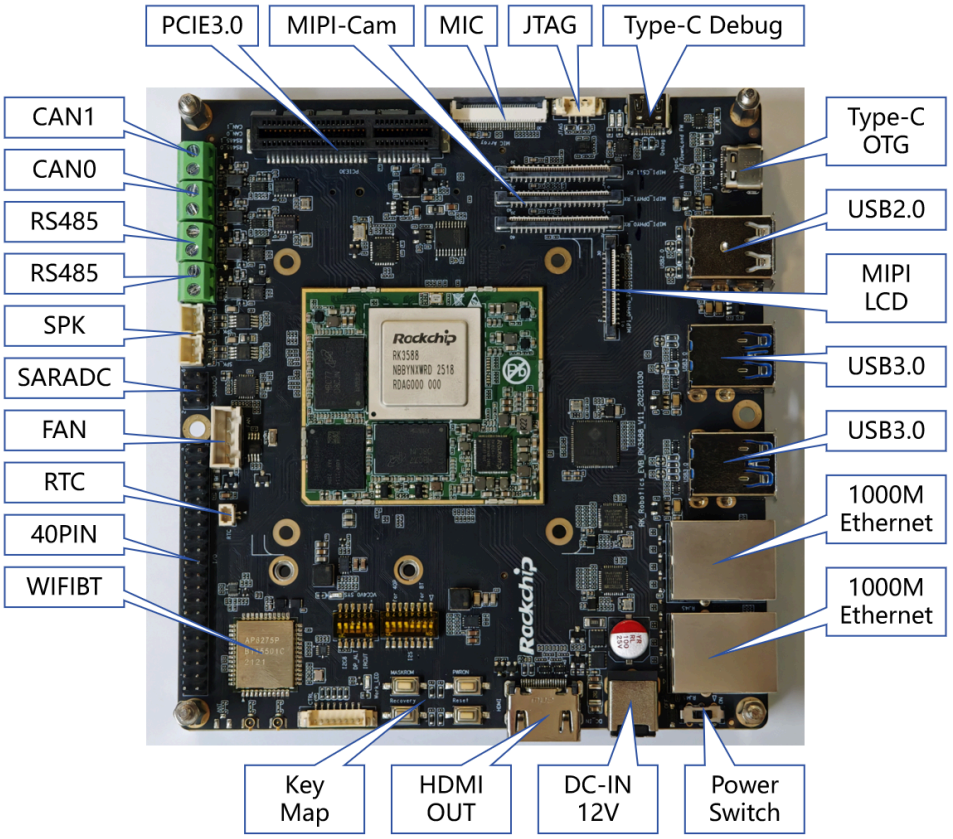
6.6 固件的打包

完成 Kernel、U-Boot、Recovery、Rootfs 的编译后，执行以下命令，将固件打包到 `output/firmware` 目录：

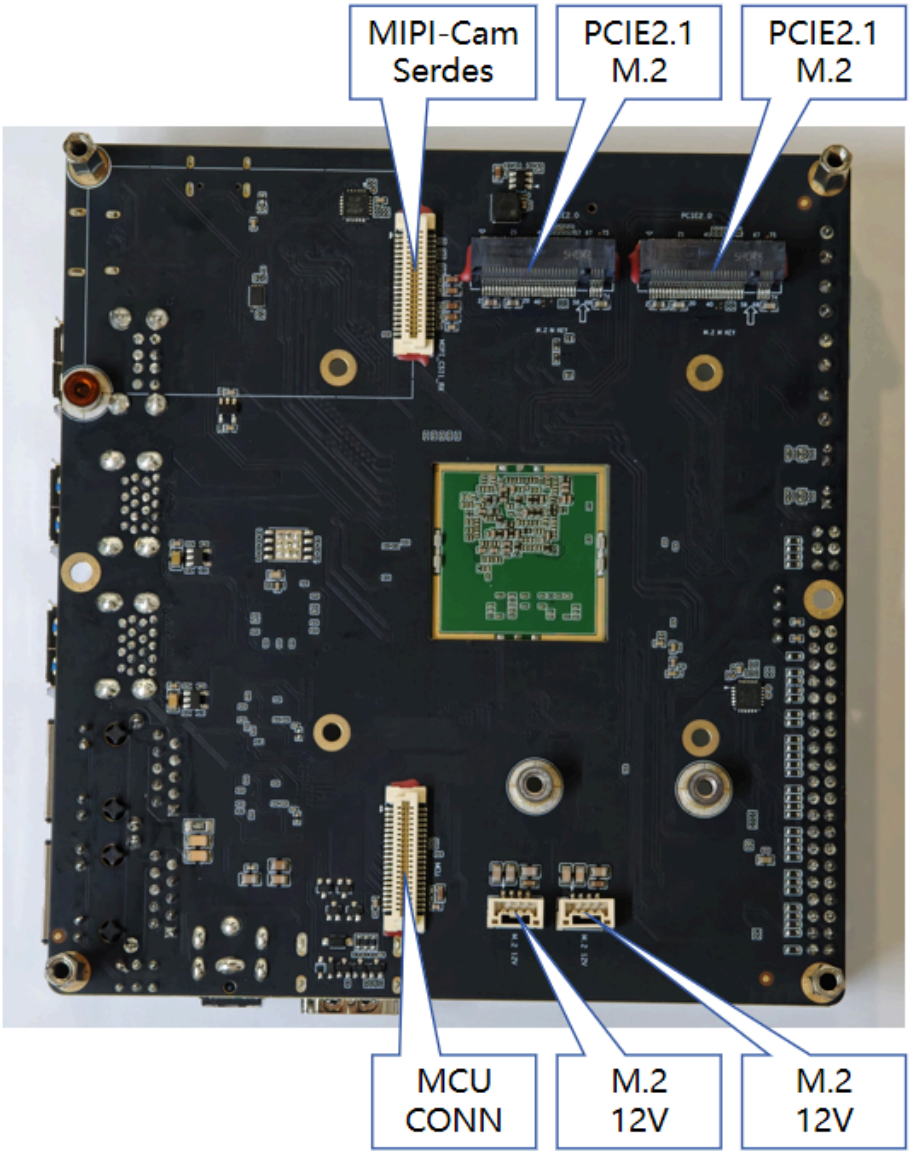
```
./build.sh firmware
```

7. 刷机说明

RK3588 Robotic EVB 开发板正面接口分布图如下：



RK3588 Robotic EVB 开发板背面接口分布图如下：

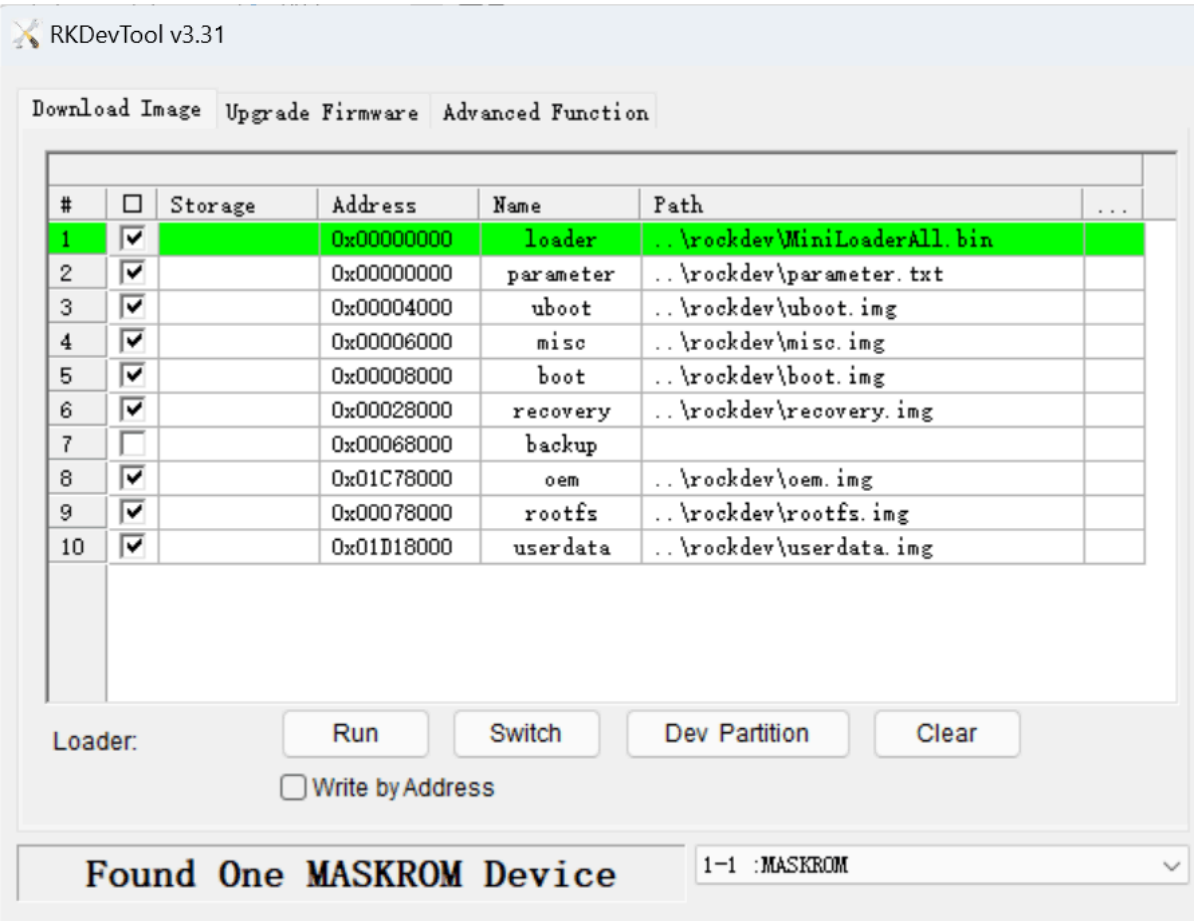


7.1 Windows 刷机说明

SDK 提供 Windows 烧写工具(工具版本需要 V3.31 或以上)，工具位于工程根目录：

```
<SDK>/tools/windows/RKDevTool
```

如下图，编译生成相应的固件后，设备烧写需要进入 MASKROM 或 BootROM 烧写模式，连接好 USB 下载线后，按住按键 `MASKROM` 不放，并按下复位键 `RST` 后松手，就能进入 MASKROM 模式，加载编译生成固件的相应路径后，点击 `执行` 进行烧写，也可以按 `recovery` 按键不放并按下复位键 `RST` 后松手进入 LOADER 模式进行烧写，下面是 MASKROM 模式的分区偏移及烧写文件。
(注意：Windows PC 需要在管理员权限运行工具才可执行)



注：烧写前，需安装最新 USB 驱动，驱动详见：

```
<SDK>/tools/windows/
└──DriverAssitant_v5.13.zip
```

7.2 Linux 刷机说明

Linux PC 的烧写工具位于 `<SDK>/tools/linux/Linux_Upgrade_Tool` 目录，工具版本需要 V2.36 或以上，请确认你的板子连接到 MASKROM/LOADER rockusb。比如编译生成的固件在 `<SDK>/rockdev` 目录下，升级命令如下：

```
sudo ./upgrade_tool ul rockdev/MiniLoaderAll.bin -noreset
sudo ./upgrade_tool di -p rockdev/parameter.txt
sudo ./upgrade_tool di -u rockdev/uboot.img
sudo ./upgrade_tool di -misc rockdev/misc.img
sudo ./upgrade_tool di -b rockdev/boot.img
sudo ./upgrade_tool di -recovery rockdev/recovery.img
sudo ./upgrade_tool di -oem rockdev/oem.img
sudo ./upgrade_tool di -rootfs rocdev/rootfs.img
sudo ./upgrade_tool di -userdata rockdev/userdata.img
sudo ./upgrade_tool rd
```

或升级打包后的完整固件：

```
sudo ./upgrade_tool uf rockdev/update.img
```

或在根目录，机器在 MASKROM 状态运行如下升级：

```
./rkflash.sh
```

7.3 系统分区说明

默认分区说明 (下面是 RK3588 Robotic EVB 分区参考)

Number	Start (sector)	End (sector)	Size	Name
1	16384	24575	4 MB	uboot
2	24576	32767	4 MB	misc
3	32768	163839	64 MB	boot
4	163840	425983	128 MB	recovery
5	425984	491519	32 MB	bakcup
6	491520	105349120	50 GB	rootfs
7	105349121	105611265	128 MB	oem
8	105611265	122142686	7.8 GB	userdata

- uboot 分区：供 uboot 编译出来的 uboot.img。
- misc 分区：供 misc.img，给 recovery 使用。
- boot 分区：供 kernel 编译出来的 boot.img。
- recovery 分区：供 recovery 编译出的 recovery.img。
- backup 分区：预留，暂时没有用。
- rootfs 分区：供 debian 编出来的 rootfs.img。
- oem 分区：给厂家使用，存放厂家的 APP 或数据。挂载在 /oem 目录。
- userdata 分区：供 APP 临时生成文件或给最终用户使用，挂载在 /userdata 目录。